Concurrency & Locking Guide

Engineering Handbook · Internal · Owner: Platform Engineering

Purpose

Our services run as many concurrent instances (Cloud Run revisions, GKE replicas, worker-service pods) behind a load balancer or pulling from Pub/Sub. Any assumption that "only one thing runs at a time" is false. This guide is the prescriptive standard for coordinating concurrent work safely: distributed mutual exclusion, idempotency, atomic updates, and consistent lock ordering. Apply it to any job or handler that has an external side effect or mutates shared state.

Standards

Distributed locks for cross-instance exclusion

A local `threading.Lock` only protects one process. When two instances could process the same job, guard the critical section with a distributed lock (for example a Redis/Memorystore lock with a TTL) *and* back it with a durable uniqueness constraint, because a lock alone can be lost on crash or TTL expiry.

```
token = str(uuid.uuid4())
# NX = only if absent; EX = auto-expire so a crash can't hold the lock forever
if not redis.set(f"lock:job:{job_id}", token, nx=True, ex=30):
   return  # another instance owns it
try:
   do_side_effect(job_id)
finally:
   # release only if we still own the lock (check-and-delete)
   if redis.get(f"lock:job:{job_id}") == token:
       redis.delete(f"lock:job:{job_id}")
```

Idempotency as the durable backstop

Make the operation idempotent so a retry or a lost lock cannot double-apply it. Require an idempotency key on mutating requests, store it with a uniqueness constraint, and return the original result on replay.

```
try:
   db.execute(
       "INSERT INTO processed(idempotency_key, result) VALUES (:k, :r)",
       {"k": key, "r": result})
except UniqueViolation:
   return load_existing_result(key)  # replay: return the first outcome
```

Atomic updates instead of read-modify-write

Never read a value into the app, modify it, and write it back under concurrency — a second request between read and write is lost. Use an atomic database update or a compare-and-swap, or a transaction with row locking.

```
-- atomic; no lost update
UPDATE accounts SET balance = balance + :delta WHERE id = :id;
```

Consistent lock ordering

When code must hold more than one lock, acquire them in one global order everywhere, add a timeout on acquisition, and keep the critical section minimal. Opposite acquisition orders in two code paths deadlock under load.

Common failures and fixes

- Two instances process the same job and both perform the side effect (double charge/email) → no cross-instance mutual exclusion; guard with a distributed lock or uniqueness constraint and make the operation idempotent with an idempotency key (BUG-1054).
- A counter or balance is wrong under concurrency → lost update from read-modify-write; use an atomic `UPDATE ... SET x=x+1` or compare-and-swap, or a transaction with row locking (BUG-1052).
- The service hangs intermittently → deadlock from inconsistent lock ordering; acquire locks in a single global order everywhere, add an acquisition timeout, and shrink the critical section (BUG-1053).
- Retried requests create duplicate records → missing idempotency key; require a key on mutating requests, store it with a uniqueness constraint, and return the original result on replay (BUG-1057).

Checklist

- Every externally-visible side effect is idempotent or guarded by a uniqueness constraint.
- Cross-instance critical sections use a distributed lock with a TTL and a safe check-and-delete release.
- Shared counters/balances updated atomically in the database, never read-modify-write in app code.
- Multiple locks acquired in one documented global order, with a timeout.
- Mutating APIs accept and enforce an idempotency key.
- Concurrency behavior covered by a test that runs the path in parallel.

Related incidents

BUG-1054, BUG-1052, BUG-1053, BUG-1057